O KERNEL LINUX: MÓDULOS, /PROC E /SYS

O adaptador USB de Wi-Fi funcionava. Depois de um `apt full-upgrade` e do reboot que ele pediu, `ip link` mostra só `lo` e `eth0`. O LED acende, o `lsusb` lista o aparelho, o journal não reclama de nada. ***O dispositivo existe; o driver, não*** *— o kernel é novo e ninguém recompilou o módulo.*

098
GUIAS LINUX

A FRONTEIRA DE PRIVILÉGIO — E AS DUAS JANELAS PARA DENTRO DO KERNEL

APLICAÇÕES
bash · ls · nmap · Firefox · seus scripts

BIBLIOTECAS DE SISTEMA
glibc · libssl · libpcap

chamada de sistema — ~350 pontos de entrada no x86-64

TABELA DE SYSCALLS
open · read · write · fork · execve · mmap · socket

ESCALONADOR
processos

MEMÓRIA
páginas e cache

VFS
ext4 · Btrfs · proc

REDE
TCP/IP · netfilter

DRIVERS DE DISPOSITIVO
quase sempre módulos `.ko` em `/lib/modules/$(uname -r)/`

HARDWARE
CPU · RAM · discos · placas de rede · USB

`/proc`
processos e estado global · `/proc/sys` é o ramo gravável

`/sys`
modelo de dispositivos · um valor por arquivo

Tudo acima da linha vermelha só sabe *pedir*; tudo abaixo executa com privilégio total. As duas caixas à direita não ocupam um byte de disco: são arquivos gerados sob demanda por funções do kernel — daí examinar e ajustar o kernel com `cat` e `echo`. `/proc` **cresceu sem plano e virou sótão** (blocos de texto para humano ler); `/sys` **nasceu com regra** — `/sys/class/net/eth0/mtu` contém exatamente `1500`, trivial em script. `lsmod` é só um formatador de `/proc/modules`.

DO HARDWARE DETECTADO AO DRIVER LIGADO — E ONDE A CADEIA ARREBENTA CALADA

HARDWARE
adaptador conectado
`lsusb · lspci -nnk`

KERNEL
emite um *uevent* com o MODALIAS
`dmesg -w`

SYSTEMD-UDEVD
casa o alias com `modules.alias`
`udevadm monitor`

MODPROBE
carrega o `.ko` e as dependências
`modinfo · dkms status`

/SYS E /DEV
`wlan0` aparece, as regras rodam
`ip link · ls /sys/class/net`

É no quarto degrau que o adaptador da abertura morre — módulo ausente, `vermagic` de outra versão ou blacklist —, **e ninguém reclama alto.** O veredito está no `lspci -nnk`: se aparece `Kernel driver in use:`, o driver está ligado; se aparece só `Kernel modules:`, o kernel *sabe* qual serviria e não conseguiu carregá-lo. **Cada versão de kernel tem o seu próprio `/lib/modules/`:** módulo compilado para o `6.12.13` é recusado pelo `6.12.20`. A cura é o **DKMS**, que guarda o *fonte* e recompila a cada kernel novo — desde que `linux-headers-$(uname -r)` esteja instalado.

NA PRÁTICA — CARREGAR, PERSISTIR E GIRAR OS BOTÕES

kali@kali: ~

```
# o modulo dummy cria placas de rede falsas:
# o jeito seguro de exercitar tudo isto
$ sudo modprobe dummy numdummies=2
$ lsmod | grep dummy
dummy     16384  0
$ cat /sys/module/dummy/parameters/numdummies
2
$ ip -brief link show type dummy
dummy0  DOWN  9a:1c:..  dummy1  DOWN  6e:04:..
# o adaptador sumiu depois do upgrade? tres comandos:
$ sudo apt install linux-headers-$(uname -r)
$ dkms status   # registrado so p/ o kernel velho
$ sudo dkms autoinstall   # recompila; reboot
# o log do kernel, o trio que resolve:
$ dmesg -T -l err,warn   # com data, so o que dói
$ journalctl -k -b -1    # o boot ANTERIOR
# quem esta de fato ligado ao dispositivo:
$ lspci -nnk | grep -A3 -i network
    Kernel driver in use: e1000e
    Kernel modules: e1000e
```

ONDE MEXER	O QUE RESOLVE
modprobe × insmod	`modprobe` lê o grafo de dependências (`modules.dep`) e resolve; `insmod` obedece ao pé da letra e falha com `Unknown symbol in module`
persistir	`/etc/modules-load.d/` carrega no boot; `/etc/modprobe.d/` guarda parâmetros e blacklist
blacklist + install	`blacklist` só barra o carregamento **automático por alias**; módulo que declare dependência ainda puxa o proibido — `install nouveau /bin/false` fecha a porta. `update-initramfs -u` , o passo mais esquecido
sysctl / /proc/sys	o mesmo endereço por dois caminhos: `net.ipv4.ip_forward` , `kernel.dmesg_restrict` , `vm.swappiness` . Persiste em `/etc/sysctl.d/`
/proc/<PID>/	de onde `ps` e `htop` tiram tudo: `cmdline` , `status` , `maps` , `fd/` , `environ` , `ns/`

Descarregar módulo não é operação inócua. `sudo modprobe -r` (ou `rmmod`) no driver de disco, de um sistema de arquivos montado ou da placa por onde passa a sua sessão SSH **derruba o serviço na hora** — e, no caso da rede, corta a própria conexão que você usaria para desfazer. O kernel recusa descarregar módulo com uso registrado (coluna `Used by` do `lsmod`), mas essa contagem **nem sempre reflete o uso real**. Ensaie em VM descartável com o `dummy` .

RESUMO RÁPIDO

- ✓ **O kernel é a única peça que executa com privilégio total.** Aplicação não acessa hardware — ela *pede*, pela chamada de sistema.
- ✓ **Monolítico, mas modular:** uma imagem serve milhares de máquinas porque carrega só o `.ko` do que encontra. **Cada kernel tem o seu** `/lib/modules/` — módulo de outra versão é recusado pela `vermagic` : a causa nº 1 de hardware que "some" depois de um upgrade.
- ✓ `modprobe` **resolve dependências,** `insmod` **não.** O que persiste vive em `/etc/modprobe.d/` e `/etc/modules-load.d/` — e blacklist sem `update-initramfs -u` simplesmente não vale no boot.
- ✓ **DKMS é a solução estrutural** para driver fora da árvore: guarda o fonte e recompila a cada kernel novo, com `linux-headers-$(uname -r)` como pré-requisito absoluto.
- ✓ `/proc` **= processos e estado;** `/sys` **= dispositivos.** `/proc/sys` é o ramo gravável e `sysctl` é a interface oficial dele.
- ✓ **O trio de diagnóstico:** `dmesg -T -w` , `journalctl -k -b -1` (o boot anterior, que o `dmesg` perdeu) e `lspci -nnk` .

NO KALI

O Kali é a distribuição em que "recompilar módulo" mais acontece. Adaptadores Wi-Fi USB com chipset Realtek (`rtl8812au` , `rtl88xxau`) são a base do trabalho com redes sem fio e vários não têm driver na árvore oficial — o caminho suportado é `apt install realtek-rtl88xxau-dkms` , e cada `full-upgrade` com kernel novo dispara a recompilação. **Modo monitor e injeção dependem do *driver*, não da placa:** `iw list` diz se o driver carregado aceita `monitor` — dois drivers do mesmo chipset divergem nisso, e é por isso que `modinfo` entra no diagnóstico de "o `airmon-ng` não coloca em modo monitor". **Ferramenta que parece com defeito costuma ser botão na posição errada:** intermediário exige `net.ipv4.ip_forward=1` ; captura promíscua exige `CAP_NET_RAW` . `/proc` **é fonte de forense *e* de pós-exploração** — `environ` , `cmdline` e `maps` de processo alheio revelam credenciais passadas por variável e por linha de comando; a contramedida é montar `/proc` com `hidepid=2` . Só em sistema próprio ou autorizado: invadir é crime (Lei 12.737/2012). **Rootkit de kernel é, tecnicamente, um módulo** — carregado como root, intercepta syscalls e esconde processos, arquivos e conexões, o que inutiliza qualquer ferramenta de user space na mesma máquina. Defesas: `kernel.modules_disabled=1` depois do boot, exigir assinatura, vigiar `lsmod` e o bit de *taint* — e investigar máquina suspeita de mídia externa, nunca do sistema em execução. **Com Secure Boot ativo,** módulo não assinado falha com `Key was rejected by service` — mensagem que nunca sugere a causa. **Na RHEL:** `kernel-devel` , `dracut -f` e assinatura MOK são a regra.